

G01 · 5분 만에 이해하는 학습 — gradient·momentum·optimizer

이 권은 “모델을 학습시킨다”가 시스템 관점에서 정확히 무슨 연산인지를, 수식을 거의 보지 않고도 그림으로 잡게 해 준다. 우리가 아는 serving에서 weight는 read-only 상수지만, 이 시리즈의 논문들 (Titans·Miras·Atlas·TNT·Nested Learning·Sleep)은 **inference** 도중에도 작은 weight를 매 token 갱신한다. 그 “갱신”의 정체가 바로 학습, 즉 gradient·momentum·optimizer 이므로, 이 셋을 물리적 직관 수준에서 이해해 두면 뒤 권들이 전부 “이미 아는 부품의 재배치”로 읽힌다. 이 권의 마지막에 놓을 반전 하나 — **optimizer**는 과거 gradient를 기억하는 작은 메모리다 — 가 시리즈 전체의 토대다.

1. 학습이란 손잡이를 돌려 “틀린 정도”를 줄이는 일

모델에는 수십억 개의 **손잡이** (parameter, 기호로 Θ)가 달려 있다. 학습이란 이 손잡이들을 조금씩 돌려, **틀린 정도**를 나타내는 숫자 하나 (loss, 기호로 $\mathcal{L}$)를 낮추는 반복 작업이다. 여기서 놓치면 안 되는 사실 하나: 수십억 개 손잡이의 모든 행동이 결국 **숫자 딱 하나**로 요약되고, 학습은 그 숫자 하나를 내리는 방향으로 전체를 움직인다.

비유 loss는 “지금 이 모델이 얼마나 틀렸나”를 재는 하나의 점수판이다. 학습 = 점수판 숫자를 낮추려고 수십억 개 손잡이를 아주 조금씩 돌리는 일. language model 이라면 이 점수는 “다음 token을 얼마나 못 맞혔나”의 평균이다.

한 번의 학습 step은 언제나 네 단계다.

1. **forward pass** — 데이터를 모델에 통과시켜 loss를 계산한다. 우리가 아는 prefill과 커널 구성이 똑같다 (GEMM과 attention의 연속).
2. **loss 계산** — 출력을 점수 하나로 줄인다. 비용은 무시할 수준.
3. **backward pass** — 그 점수를 기준으로 “각 손잡이를 어느 쪽으로 얼마나 돌려야 하나”를 계산한다 (2절). FLOPs는 forward의 약 2배.
4. **optimizer step** — 3에서 나온 지시와 optimizer의 내부 메모리를 읽어 손잡이를 실제로 돌린다 (4절).

이 loop를 우리가 매일 돌리는 serving loop 옆에 놓으면 대응이 바로 보인다.

항목	serving loop (우리 세계)	training loop (이 권)
weights	read-only (mmap된 checkpoint)	매 step 읽고-고치고-쓰기
반복 단위	request / token	step / batch
지속되는 상태	KV cache (request 수명)	optimizer 메모리 (학습 전체 수명)
주 커널	forward GEMM + attention	forward GEMM + backward GEMM 2종 + elementwise step

항목	serving loop (우리 세계)	training loop (이 권)
rooeline	prefill=compute-bound, decode=bandwidth-bound	fwd/bwd=compute-bound, optimizer step=bandwidth-bound

낮선 칸은 딱 두 개다: **backward pass** 와 **optimizer** 상태. 이 권의 나머지가 그 두 칸을 채운다.

시스템 모델링 관점 학습 step 은 우리가 아는 두 커널의 조합으로 분해된다. forward/backward 는 **prefill** 을 다했고 (두꺼운 GEMM, compute-bound), optimizer step 은 **decode** 를 다했다 (거대한 상태를 한 번 훑는, bandwidth-bound). 이 시리즈가 하는 일은 이 표의 왼쪽 열과 오른쪽 열을 **한 프로세스로 합치는 것** — decode loop 안에서 작은 network 의 weight 가 매 token 갱신된다. “serving 중 weight 불변” 이라는 우리 세계의 불변식이 여기서 절반 폐기된다.

2. gradient = 경사, backward pass = 틀린 정도를 거꾸로 흘리기

gradient: 어느 쪽이 내리막인가

gradient 는 손잡이마다 하나씩 붙는 숫자로, “이 손잡이를 아주조금 올리면 loss 가 얼마나 변하나” 라는 **민감도**다. 부호와 크기를 함께 담는다: 부호는 “올릴까 내릴까”, 크기는 “얼마나 영향이 큰가” .

직관 안개 낀 산에서 발밑 경사만 느끼며 내려가는 상황. gradient 는 “지금 서있는 자리에서 어느 쪽이 오르막인가” 를 손잡이별로 알려주는 화살표다. 내려가려면 화살표 반대로, 정해진 보폭만큼 한 걸음 옮기면 된다. 이게 **gradient descent** 의 전부다.

$$\Delta \Theta = - \eta \nabla_{\Theta} \mathcal{L}$$

기호 풀이 Θ = 모델의 손잡이 (parameter) 전체. $\mathcal{L}$ = loss(틀린 정도, 숫자 하나). $\nabla_{\Theta} \mathcal{L}$ = gradient(손잡이별민감도 화살표, Θ 와 같은 크기의 텐서). η (에타) = **learning rate**, 한 걸음의 보폭. $\Delta \Theta$ = 이번 step 에 손잡이를 얼마나 움직일지.

이 식은 이런 뜻. “민감도 화살표의 **반대 방향**으로, 보폭 η 만큼 손잡이를 움직여라.” 반대 방향인 이유는 loss 를 **내리고** 싶기 때문이다.

backward pass: 왜 거꾸로 흘리는가

문제는 gradient 를 어떻게 싸게 구하느냐다. 손잡이 하나를 살짝 흔들고 forward 를 다시 돌려 민감도를 재면, 손잡이가 10^9 개니까 forward 를 10^9 번 돌려야 한다 — 논외다.

backpropagation(줄여서 backward pass)은 forward 한 번 + 거꾸로 한 번으로 모든 손잡이의 gradient 를 한꺼번에 얻는다.

비유 팀이 낸 결과가 틀렸을 때, “최종 오차”를 맨 뒤 단계부터 앞 단계로 거꾸로 배분하는 책임 추적과 같다. 각 단계는 아래 (뒤) 단계에서 “네 탓이만큼”이라는 오차 신호를 받아, (a) 자기 손잡이를 얼마나 돌릴지 계산하고 (b) 그 오차를 다시 한 단계 더 뒤로 넘긴다. 이 “오차를 거꾸로 흘리기”가 backward pass 다.

이 과정에서 한 layer 의 손잡이 gradient 는 언제나 같은 모양으로 나온다. 이권에서 딱 하나 기억할 수식이다.

$$dW = \delta x^T$$

기호 풀이 W = 한 layer 의 weight 행렬. dW = 그 layer 손잡이의 gradient (= “이 손잡이들을 얼마나 돌릴지”). x = 그 layer 에 들어온 입력. δ (델타) = 그 layer 에서 뒤로 흘러온 오차 신호. $x^T = x$ 를 가로로 눕힌 것 (전치). δx^T = 두 벡터의 **outer product** (바깥곱), 즉 세로 벡터 $\times$ 가로 벡터로 만든 행렬.

이 식은 이런 뜻. “어떤 손잡이를 얼마나 돌릴지 = (그 층에 들어온 입력) $\times$ (그 층의 오차)의 바깥곱.” 학습이 weight 에 가하는 모든 변화는 결국 이런 바깥곱들의 합이다.

시스템 모델링 관점 이 shape 하나가 시리즈 전체를 관통한다. $dW = \delta x^T$ 는 우리가 아는 KV cache write $v_t k_t^T$ (value $\times$ key 의 바깥곱으로 이어붙이기)와 정확히 같은 **GEMM family** 다. 즉 “학습의 gradient 만들기” 커널과 “memory 에 써넣기” 커널이 shape 수준에서 동일하다. batch B 개를 쌓으면 $(d' \times B)(B \times d)$ GEMM 하나 — 바깥곱 B 개를 쌓는 rank- B 누적이다. 이 동일성 덕분에 학습용으로 존재하는 모든 커널 기법 (tiling, tensor-core, rank 누적)이 이 시리즈의 inference 에 그대로 이식된다. backward 가 forward 의 약 2 배인 이유도 여기 있다: forward 가 GEMM 1 개일 때 backward 는 같은 크기 GEMM 2 개 (오차 전파용 + gradient 생성용) 다.

주의 backward 는 FLOPs 만 드는 게 아니다. dW 를 계산하려면 forward 때의 입력 x 를 전부 붙들고 있어야 한다 (activation memory). 큰 모델에서는 이게 weight 보다 먼저 메모리를 바닥낸다. 표준 대응은 **recomputation** (일부를 버리고 backward 중 다시 계산) — FlashAttention 이 softmax 중간값을 버리고 재계산하는 것과 같은 “FLOPs 내고 bytes 사기” 거래다.

3. loss surface = 골짜기 지형, 왜 보폭 하나로는 부족한가

loss 를 손잡이 개수만큼의 차원을 가진 지형으로 상상하자. gradient descent 는 이 지형을 발밑 경사만 보고 내려간다. optimizer 설계를 지배하는 건 지형의 전체 모양이 아니라 국소적으로 방향마다 경사가 얼마나 다른가다.

비유 좁고 긴 골짜기를 상상하자. 골짜기를 가로지르는 방향은 벽이 가팔라서 조금만 움직여도 경사가 확 뒤집힌다. 골짜기를 따라가는 방향은 완만해서 한참 가도 경사가 그대로다. 이상적 보폭이 방향마다 다른데 보폭 η 는 하나뿐이다. 가파른 방향에 맞추면 완만한 방향에서 기어가고, 완만한 방향에 맞추면 가파른 방향에서 튕겨 진동한다. 이 딜레마를 ill-conditioning 이라 부른다.

핵심 이 절만 기억하면 4 절 전체가 예습된다. step 의 품질은 gradient 만으로 정해지지 않는다. gradient 를 “어떤 상태 (과거 기억) 와 어떤 기하 (방향별 스케일) 로 가공하느냐” 가 품질을 정한다. 그 가공기가 optimizer 이고, 4 절의 optimizer 들은 전부 이 골짜기 문제에 대한 서로 다른 응답이다 — 진동을 평균으로 지우거나 (momentum), 좌표별로 보폭을 다시 재거나 (AdamW), 방향별 크기를 고르게 퍼거나 (Muon).

4. optimizer 동물원 — SGD, momentum, weight decay, AdamW, Muon

여기서 optimizer 를 우리가 아는 방식 그대로, (state, update, cost) 를 갖는 객체로 본다. attention 을 KV cache(state) + attention 식 (update) + roofline(cost) 로 보듯이 말이다.

기호 풀이 (이 절 공통). g_t = step t 의 gradient. η = 보폭 (learning rate). β (베타) = momentum 감쇠상수 (보통 0.9). m_t = momentum 메모리 (buffer). h_t = gradient 제공의 누적 (AdamW 의 둘째 메모리). λ (람다) = weight decay 강도. “state 크기” 는 손잡이 하나당 fp32 기준 byte.

객체	state (크기)	한마디로	step 성격
SGD	없음 (0 B)	한 걸음씩 내리막	elementwise, BW-bound
+ momentum	m_t (4 B)	무거운 공 (관성)	elementwise
+ weight decay	없음 (0 B)	0 쪽으로 살살 당기기	elementwise
AdamW	m_t, h_t (8 B)	걸음 크기 자동 조절	elementwise
Muon	m_t (4 B)	방향을 더 똑똑하게	행렬당 GEMM ~12

4.1 SGD — 한 걸음씩내리막

직관 2 절의 gradient descent 그 자체. 메모리 없이, 지금 이 순간의 경사만 보고반대로 한 걸음. $\Delta\Theta = -\eta g_t$. 가장 단순하고, 이 시리즈의 **기준선**이다: 뒤에 나올 TTT-Linear 와 delta rule 의 “매 token 학습” 이 정확히 이 stateless SGD 다. Titans 의 기여는 여기에 **메모리**를 추가한 것으로 요약된다.

4.2 momentum — 무거운 공 (관성)

직관 가벼운 공은 경사가 바뀔 때마다 휘청인다. 무거운 공은 관성이 있어, 최근에게속 굴러온 방향을 유지하며 진동을 스스로 지운다. momentum 은 gradient 에 이 관성을 준다 — “최근 놀란 방향의 관성”.

$$m_t = \beta m_{t-1} + g_t, \quad \Delta\Theta_t = -\eta m_t$$

기호 풀이 m_t = momentum 메모리, “지금까지 굴러온 방향”의 누적. β = 과거를 얼마나 유지할지 (0.9 면 과거의 90%를 남김). g_t = 이번 gradient. 나머지는 위와 같다.

이 식은 이런 뜻. “이번 이동 방향 = (과거 방향의 90%) + (이번 gradient).” 풀어 쓰면 m_t 는 과거 gradient 들의 **지수가중합**이다 — 최근 것일수록 크게, 오래된 것일수록 작게 섞인다. 골짜기를 가로지르는 진동은 부호가 번갈아 나와 서로 상쇄되고, 따라 내려가는 일관된 방향은 최대 $1/(1 - \beta)$ 배까지 증폭된다.

시스템 모델링 관점 momentum 식은 **linear recurrence** 다 — 우리가 아는 linear-RNN 상태 갱신, RetNet 의 상수 decay, scan 커널이 처리하는 점화식과 **정확히 같은 대수**다. state 는 손잡이당 숫자 하나 (m_t), 갱신은 elementwise, 비용은 SGD 의 약 2 배 트래픽. 이 recurrence 가 linear 이기때문에 chunk 안에서 **associative scan** 으로 병렬 계산된다 (우리의 prefix-sum 커널이 여기서 재취업한다). Titans 는 이 m_t 를 그대로 inner loop 로 옮겨 “past surprise” S_t 로 쓴다.

4.3 weight decay — 손잡이를 0 쪽으로 살살당기기

직관 매 step 손잡이 값에 0.99 같은 수를 곱해, 쓰이지 않는 손잡이를 서서히 0 으로 끌어당긴다. 과적합을 막는 장치. $\Theta \leftarrow (1 - \eta\lambda)\Theta - (\text{optimizer의 이동})$.

기호 풀이 λ = decay 강도. $(1 - \eta\lambda)$ = 매 step 살아남는 비율 (예: 0.99). 이 인자를 곱하는 순간, 초기 손잡이 값조차 시간이 지나면 **지수적으로 사라진다**.

시스템 모델링 관점 $(1 - \eta\lambda)$ 는 사실 **retention gate** 의 원형이다. cache 에서 오래된 항목을 지수적으로 밀어내는 “학습된 eviction” 의 가장 원시적 형태 — TTL(time-to-live) 이 걸린 cache entry 와 같다. 접근이 없어도 매 step 10%씩 증발하는 식. 이 상수 인자를

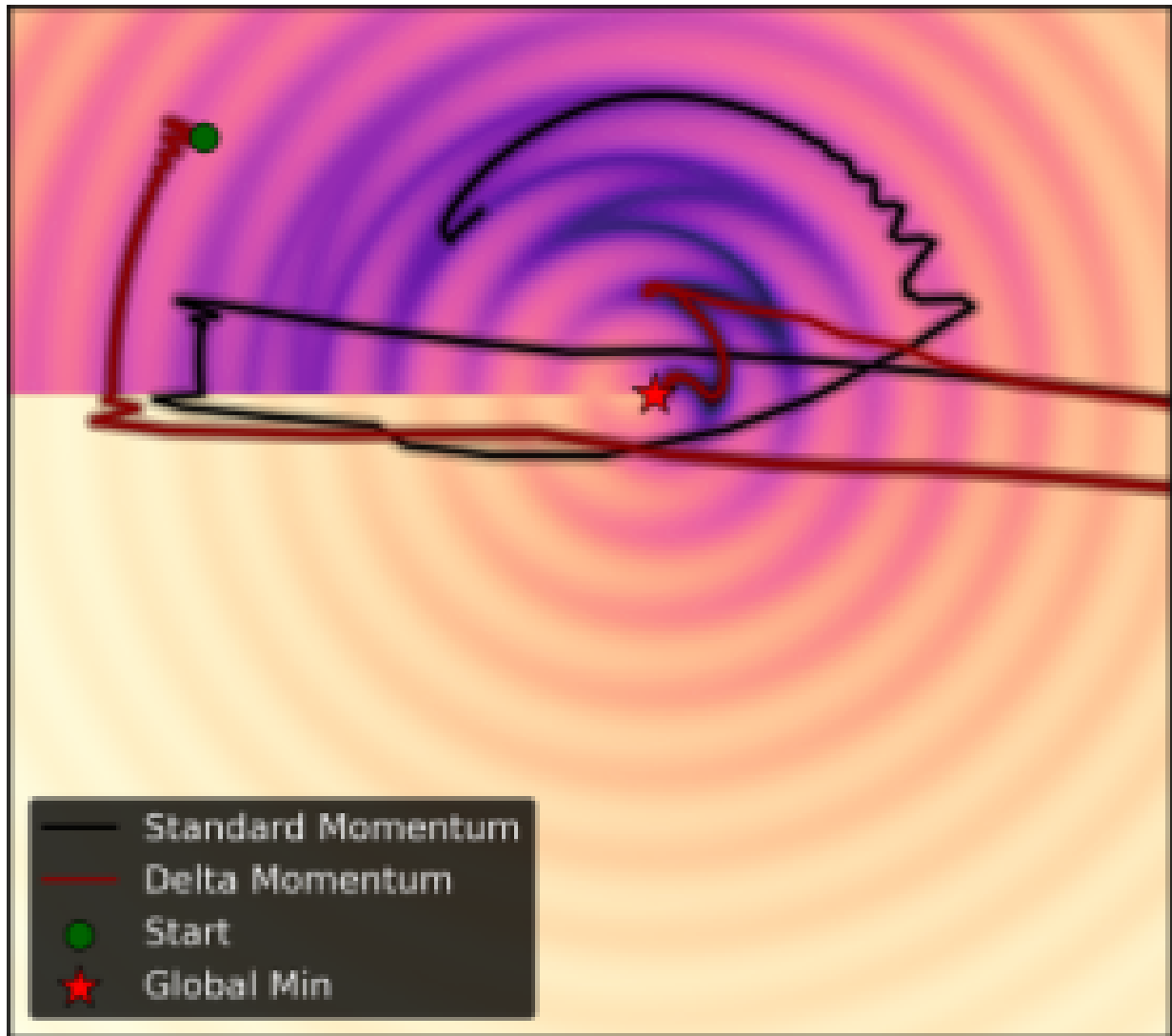


그림: 같은 골짜기 지형 (밝을수록 loss 낮음) 을 표준 momentum(검정) 과 delta momentum(빨강) 이 각각 어떻게 내려가는지 비교. 표준 momentum 은 관성 때문에 크게 휘돌지만, 관성이 결국 국 최소점 (빨간별) 으로 데려간다. 출처: Behrouz et al., Nested Learning (arXiv:2512.24695)
Fig.4 — 원저자 그림

token마다 계산되는 함수 α_t 로 승격시킨 것이 Titans·Miras의 forgetting/retention 메커니즘이다. **weight decay**는 이미 언제나 **memory** 관리연산이었다.

4.4 AdamW — 걸음 크기를 자동조정

직관 손잡이마다 gradient 스케일이 천차만별이다 (어떤 embedding row는 크게, 깊은 layer는 작게). SGD의 단일 보폭으로는 감당이 안 된다. AdamW는 **각 손잡이의 최근 gradient 크기로 그 손잡이의 보폭을 나눠** 정규화한다 — 늘 크게 흔들리는 손잡이는 억제하고, 늘 작게 흔들리는 손잡이는 키워, 모두가 비슷한 크기로 움직이게 한다.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad h_t = \beta_2 h_{t-1} + (1 - \beta_2) g_t \odot g_t, \quad \Delta \Theta_t = -\eta \frac{m_t}{\sqrt{h_t} + \epsilon}$$

기호 풀이 m_t = gradient 방향의 평균 (1차 moment, momentum과 같은 것). h_t = gradient 제곱의 평균 (2차 moment, 각 손잡이의 최근 크기 추정). $\odot$ = 원소별 곱. β_1, β_2 = 두 평균의 감쇠 상수. $\epsilon = 0$ 으로 나누기 방지용 아주 작은 수. (bias correction 항은 초반 보정용 산수라 생략.)

이 식은 이런 뜻. “이동 = (평균 방향) ÷ (그 손잡이의 전형적 크기).” 나눗셈 덕분에 모든 손잡이가 대략 $\pm \eta$ 스케일로 움직인다. 그래서 Adam은 “**부호에 가까운 (sign-ish) 이동 + 손잡이별 보폭**”으로 요약된다. AdamW = 이 식 + decoupled weight decay이며, LLM pretraining의 기본값이다.

시스템 모델링 관점 state는 손잡이당 메모리 2개 (m_t, h_t , fp32 8 B). 갱신은 여전히 GEMM 없는 순수 elementwise pass — 트래픽은 SGD의 약 3배, roofline은 decode와 같은 극단적 bandwidth-bound. **학습이 서빙보다 HBM을 몇 배 먹는 이유의 절반이 이 optimizer 상태다.** 7B 모델을 bf16 weight + fp32 master + Adam m + Adam h 로 두면 손잡이당 14 byte, 약 98 GB가 gradient/activation을 세기도 전에 상주한다.

4.5 Muon — 방향을 더 똑똑하게

직관 AdamW가 좌표축마다 크기를 고르게 폈다면, Muon은 임의의 방향 (특이값)마다 고르게 편다. momentum 행렬은 보통 소수의 지배적 방향에 힘이 몰려 있어, 그대로 쓰면 이동이 사실상 저 rank가 된다. Muon은 그 행렬을 “가장 가까운 방향-보존·크기-균일 행렬”로 갈아 끼워, 희귀하지만 유효한 방향까지 살려 모든 방향에 고르게 쓴다.

기호 풀이 NS_κ = **Newton-Schulz 반복**, 행렬의 방향 (특이벡터)은 그대로 두고 크기 (특이값)만 1 근처로 밀어붙이는 다항식 연산을 κ 번 (κ =카파, 보통 5) 반복. $\|m_t\|_F$ = 행렬 크기 정규화. 이동 = $-\eta \text{NS}_\kappa(m_t / \|m_t\|_F)$.

시스템 모델링 관점 여기서 성격이 질적으로 바뀐다. Muon의 state는 momentum 하나 (Adam 의절반) 지만, 갱신이 **elementwise**가 아니라 **행렬 shape**의 GEMM 10-15 개다. 즉 optimizer step을 bandwidth-bound에서 **tensor core**가 도는 **compute 연산**으로 옮긴다 — “state를 덜 쓰고 compute를 더 쓴다”는 거래. Atlas는 이 Muon을 **inner loop의 optimizer**로 인식하며, 그 순간 반복 횟수 κ 는 decode 중 chunk마다 지불하는 “test-time compute” knob이 된다 (반복을 늘리면 더 나은 memorization을 산다).

5. 핵심 반전 — optimizer는 “과거 gradient를 기억하는 작은메모리” 다

지금까지 optimizer를 “손잡이를 돌리는 도구”로 봤다. 이제 시각을 뒤집는다. 이 반전이 이 시리즈 6편 전체의 토대다.

핵심 momentum 메모리 $m_t = \beta m_{t-1} + g_t$ 를 다시 보라. 이것은 “과거 **gradient**들을 지수가중으로 압축해 담은 작은 메모리”다. optimizer는 도구가 아니라, gradient 스트림을 읽어 요약해 계속 갱신하는 작은 학습기다. 그러면 자연스러운 질문이 생긴다 — 이 메모리의 용량은? 반감기는? 무엇을 잊는가?

- **momentum = 반감기 짧은 메모리.** $\beta = 0.9$ 면 누적 기여의 50% 이상이 최근 gradient 7개에, 99% 이상이 최근 44개에 몰린다. momentum은 최근만 또렷이 기억하는 메모리다.
- **weight decay = TTL cache.** $(1 - \eta\lambda)$ 곱하기는 “접근없으면 서서히 증발”하는 cache eviction 그 자체 (4.3 절).
- **AdamW = 좌표별 요약 메모리.** Nested Learning은 element-wise ℓ_2 목적을 놓으면 그 최적 associative memory가 정확히 Adam의 $m/\sqrt{h}$ 구조로 떨어짐을 보인다. 즉 Adam은 “gradient 스트림을 좌표별로 요약하는 메모리”의 한 최적점이다.

시스템 모델링 관점 optimizer 상태를 memory로 읽는 순간, 회계가 바뀐다. KV cache가 **request** 수명의 상태이듯 optimizer 상태는 **학습-run** 수명의 상태다 — 차이는 수명과 소유자뿐. 이 시리즈가 하는 일은 그 소유자를 한 번 더 바꾸는 것이다: momentum 메모리 S_t 가 **per-session** 상태가 되어 decode loop 안으로 들어온다. 그 순간 “optimizer 상태 회계”는 학습 클러스터의 문제가 아니라 **serving의 session cache sizing** 문제가 된다. 상태 크기는 $d \times d$ 행렬이면 손잡이 d^2 개, 토큰당 연산은 $\text{read}(\text{GEMV}) 1 + \text{write}(\text{rank-1 바깥곱}) 1$ 이 기본이고, momentum·retention을 붙이면 상태가 더 붙는다.

6. inner loop vs outer loop — 그때그때 배우기 vs 공부법을 미리 배우기

마지막 구조 하나. 이 시리즈에는 두 개의 학습이 겹쳐 있다.

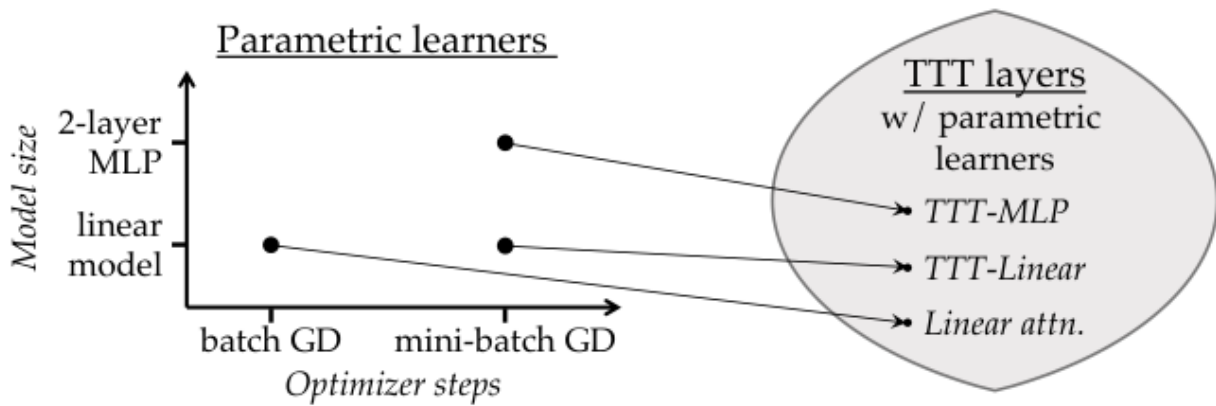


그림: “parametric learner 는 두 가지를 정해야 한다 — 모델 (무엇을기억하는 그릇) 과 optimizer(어떻게 갱신하는가)”. 세로축=모델크기 (선형→ 2-layer MLP), 가로축=optimizer step(batch GD → mini-batch GD). 이 두 축의 선택이 곧 TTT-Linear·TTT-MLP·Linear attention 같은 layer 가된다. 출처: Sun et al., Learning to (Learn at Test Time) (arXiv:2407.04620)
Fig.8 — 원저자 그림

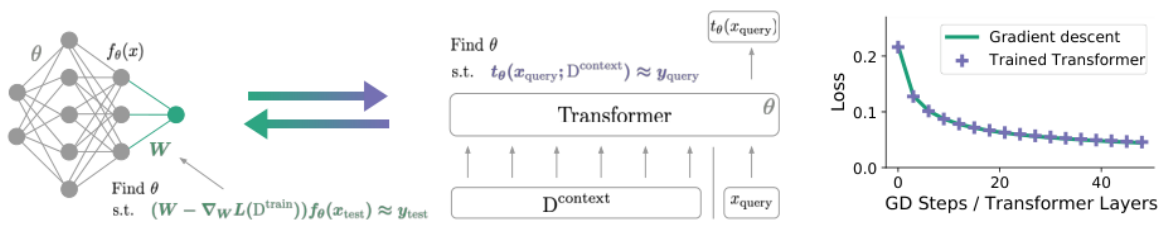


그림: “gradient 기반 최적화 = attention”이라는 가설의 그림. (왼쪽) 작은 network를 gradient 한 step 갱신하는 것과 (가운데) transformer가 문맥을 읽어 답을 내는 것이 (오른쪽) 같은 곡선을 그린다 — GD step 수를 늘리는 것과 transformer layer를 쌓는 것이 같은 곡선으로 loss를 낮춘다. 출처: von Oswald et al., Transformers learn in-context by gradient descent (arXiv:2212.07677)
Fig.1 — 원저자 그림

비유 outer loop는 시험 전에 “공부하는 법 자체”를 익혀 두는 것 — 어떤 문제엔 어떻게 접근하고, 새 자료가 오면 얼마나 세게 외울지를 미리몸에 익힌다. **inner loop**는 시험장에서 문제지를 읽으며**그때그때 배우는 것** — 방금 본 정보를 즉석에서 외워 답에 쓴다. 우리 세계로 옮기면: outer loop는 pretraining(끝나면 멈춤), inner loop는 serving 중 매 token 도는 학습이다.

핵심은 **함수와 값의 분리**다. inner loop의 보폭·momentum·retention 같은 hyper-parameter는 사람이 고르지 않는다. outer loop가 이들을 **token의 함수**로 학습한다.

기호 풀이 Θ = slow weights(outer가 학습, serving 중 동결). W_t = fast weights(inner의 memory 상태, 매 token 갱신, request마다 생성·소멸). $\eta_t = \eta(x_t; \Theta)$ = token x_t 마다 값이 바뀌는 inner 보폭 — 그 함수는 Θ 의 일부라 pretraining이 학습하고, 그 값은 serving 중 매 token 평가된다. W_{init} = memory의 학습된 초기 상태.

구성 요소	소속	움직이는 시점
projection W_K, W_V, W_Q gate 함수 $\eta(\cdot), \beta(\cdot), \alpha(\cdot)$	Θ (outer) Θ (outer)	9 pretraining 만; serving 동결 함수는 pretraining 만; 값은 매 token 평가
memory 초기 상태 W_{init}	Θ (outer)	pretraining 만; serving 에서

만드는 게 아니라, attention 이 **암묵적 · 얇게** 하던 일을 **명시적 · 깊게** (진짜 memory 상태 W_t 로 압축, momentum · 학습된 gate 추가) 만드는 프로젝트다.

시스템 모델링 관점 serving 은 두 loop 중 **하나만** 가져간다. 배포 후 outer loop 는 사라진다 — ∇_{Θ} 도, Adam 메모리 m_t, h_t 도, 웨적 저장도 없다. 대신 inner loop 가 decode 안으로 들어온다: 매 token, 작은 memory net 의 forward + backward + update. 이때 backward 는 autograd 가 아니라 **손으로 유도된 고정 커널**이다 (서빙 스택엔 autograd 가 없으므로). 배포물에 들어가는 것은 Θ 전부 + W_{init} 이고, request 마다 새로 할당하는 것은 W_t (와 momentum S_t) 다. per-request 상태가 shared-weight batching 을 깨뜨린다는 함의가 이 시리즈 시스템 파트의 핵심 긴장이다.

요약 - 학습 = 손잡이 (Θ) 를 돌려 $\text{loss}(\mathcal{L}$, 숫자 하나) 를 낮추는 반복. 한 step = forward ($\approx$ prefill) + backward ($\approx$ forward 의 2 배) + optimizer step ($\approx$ decode, bandwidth-bound). - gradient = 손잡이별 “어느쪽이 내리막” 화살표. backward pass = 오차를 뒤로 흘려 각 손잡이를 얼마나 돌릴지 계산. 그 결과는 언제나 바깥쪽 $dW = \delta x^\top$ — **memory write $vk^\top$ 와 같은 GEMM**. - optimizer 동물원: SGD(메모리 없음) → momentum(관성, 메모리 1 개, linear recurrence) → weight decay(0 쪽으로 당기기 = retention 의 원형) → AdamW(손잡이별 보폭자동 조절, 메모리 2 개) → Muon(방향별 크기 균일화, GEMM 위주). - **핵심 반전**: optimizer 는 도구가 아니라 **과거 gradient 를 기억하는 작은 메모리**다. 이 관점이 momentum S_t 를 serving 의 per-session 상태로 옮기는 시리즈 전체의 토대다. - inner loop(serving 중 그때그때 배우기) vs outer loop(pretraining 에서 공부법 배우기). “learning rate 가 학습된다” 는 함수 (outer) 와 값 (inner) 을 나눠 답한다.

다음 권 예고 — G02 「빌딩블록」에서는 이 optimizer-as-memory 관점을 실제 layer 로 조립한다: associative memory(바깥쪽으로 쓰고 곱셈으로 읽는 $d \times d$ 행렬), delta rule(= 1-step SGD 로 쓰는 memory), 그리고 linear attention 을 “결함 있는 associative memory” 로 다시 읽는다.